Разбор кода ЛР2

Этот документ объясняет, как проект отслеживает плоский объект, где выполняются основные вычисления и как повторить эксперимент. Код написан на Python и использует готовые функции OpenCV. Обучение модели не требуется.

1. Что поступает на вход и что получается

Вход — видеофайл. Первый кадр должен показывать объект крупно и достаточно резко. В тестовых роликах весь первый кадр используется как образец. Для собственного видео можно выделить объект мышью через `--select` или задать `--roi X Y W H`.

Выход — видео `tracked.mp4` с рамкой и подписью, изображение образца `template.png`, покадровая таблица `metrics.csv` и сводка `summary.json`.

Объект — плоская репродукция, открытка или обложка. Гомография описывает перспективное изменение плоскости. Она не моделирует произвольную деформацию ткани или объёмного объекта.

2. Основные понятия

Ключевая точка — характерная деталь изображения, которую можно сопоставить с деталью другого кадра. **Дескриптор** — числовое описание её окрестности. SIFT вычисляет такие точки и описания. В отличие от координат, дескриптор описывает внешний вид детали и позволяет искать её после перемещения.

Соответствие — пара «точка образца и точка текущего кадра». Некоторые пары ошибочны. **Inliers** — пары, согласующиеся с найденным геометрическим преобразованием. **Outliers** — выбросы.

Гомография H — матрица 3 × 3, преобразующая координаты одной плоскости в её изображение на другом кадре. Для точки (x, y) сначала вычисляются:

```
u = h11*x + h12*y + h13
v = h21*x + h22*y + h23
z = h31*x + h32*y + h33
x_new = u / z
y_new = v / z
```

Деление на `z` позволяет описать перспективу: дальняя сторона прямоугольника может выглядеть короче ближней. Для определения H нужны минимум четыре пары в невырожденном расположении. Программа требует больше подтверждений, чтобы уменьшить риск случайной рамки.

Оптический поток LK ищет, куда переместилась небольшая окрестность точки между соседними кадрами. Он опирается на плавность движения и похожую локальную яркость. Это дополнительный способ сопровождения, когда объект слишком мал или размыт для надёжного сопоставления с первым кадром.

3. Устройство main.py

Импорты

`argparse` разбирает командную строку. `Path` работает с путями. `csv` и `json` сохраняют измерения. `perf_counter` измеряет длительность вычислений. `cv2` выполняет обработку изображений и видео. `numpy` хранит координаты и матрицы, проверяет числовые значения и вычисляет статистику.

resize_frame

```
h, w = frame.shape[:2]
if w > max_width:
    frame = cv2.resize(frame, (max_width, round(h * max_width / w)))
```

Если кадр слишком широкий, уменьшаем его с сохранением пропорций. Изменение высоты рассчитывается по тому же коэффициенту, что и изменение ширины. Маленькие кадры не увеличиваем: увеличение не создаст новых деталей.

По умолчанию предел ширины 960. В тестах кадры 640 × 360, поэтому они не меняются. Для видео с телефона это ограничивает вычислительные затраты. Координаты ROI относятся к уже уменьшенному кадру, который показывается при выборе объекта.

PlanarTracker и его состояние

Класс хранит постоянный образец и временные данные предыдущего успешного кадра. Методы класса отделяют геометрию от чтения и сохранения файлов.

Поле	Назначение
`keypoints`, `descriptors`	SIFT-точки и описания постоянного образца
`corners`	Четыре угла образца в его собственной системе координат
`previous_gray`	Предыдущий кадр в оттенках серого
`previous_points`	Точки, подготовленные на предыдущем успешном кадре
`original_points`	Координаты этих же точек в системе исходного образца

Конструктор PlanarTracker

```
self.sift = cv2.SIFT_create(nfeatures=2000, contrastThreshold=0.02)
self.matcher = cv2.FlannBasedMatcher(
    dict(algorithm=1, trees=5), dict(checks=50))
```

SIFT находит до примерно заданного числа сильных деталей. `contrastThreshold` отсекает слишком слабые. FLANN ускоряет поиск похожих вещественных дескрипторов; здесь используется индекс KD-tree. Эти параметры не требуют обучения и фиксированы для всех трёх тестов.

Образец переводится из BGR в оттенки серого: цвет для SIFT не нужен. `detectAndCompute` одновременно находит точки и вычисляет дескрипторы. Если точек меньше минимального числа совпадений, программа сразу сообщает об ошибке вместо попытки отслеживать пустую область.

Углы задаются в порядке: верхний левый, верхний правый, нижний правый, нижний левый. Максимальные индексы равны `w - 1` и `h - 1`, поскольку индексация начинается с нуля. Тип `float32` и форма массива `N × 1 × 2` подходят функциям OpenCV, работающим с точками.

match_template и поиск SIFT

Сначала SIFT вычисляется на текущем кадре. Если дескрипторов нет или их меньше двух, подобрать двух кандидатов невозможно: возвращается отсутствие оценки.

```
pairs = self.matcher.knnMatch(self.descriptors, descriptors, k=2)
good = [pair[0] for pair in pairs if len(pair) == 2
        and pair[0].distance < self.ratio * pair[1].distance]
```

`k=2` означает двух ближайших кандидатов для каждого дескриптора образца. Маленькое расстояние означает большее сходство. Проверка `len(pair) == 2` защищает от неполной пары.

Условие `d1 < 0.75*d2` — ratio test. Если расстояния 30 и 100, совпадение принимается: 30 < 75. Если 70 и 80, оно отклоняется: 70 не меньше 60. Во втором случае два кандидата похожи почти одинаково, поэтому выбор неоднозначен.

Затем пары сортируются по расстоянию и через `setdefault` оставляется только одна пара на `trainIdx`. Это не позволяет одной найденной точке кадра несколько раз подтверждать разные точки образца. `queryIdx` указывает на точку образца, `trainIdx` — на точку кадра.

Если осталось меньше `min_matches=12` пар, SIFT-оценка отклоняется. По остальным формируются два массива координат:

```
src = np.float32([self.keypoints[m.queryIdx].pt for m in good])
dst = np.float32([keypoints[m.trainIdx].pt for m in good])
```

В полном коде они приводятся к форме `N × 1 × 2`. Порядок пар одинаковый: `src[i]` соответствует `dst[i]`.

```
matrix, mask = cv2.findHomography(src, dst, cv2.RANSAC, 5.0)
```

RANSAC пробует преобразования по поднаборам пар и ищет согласованное решение. `5.0` — допустимая ошибка перепроецирования в пикселях рабочего кадра. Маска сообщает, какие пары поддержали матрицу. `mask.sum()` даёт число inliers.

Принимаются минимум 12 inliers и доля inliers не меньше 0.5. Это эвристические пороги данного проекта, а не математическая гарантия точности. Метод возвращает матрицу и статистику, а проверка формы рамки выполняется отдельно.

valid_corners и проверка рамки

```
corners = cv2.perspectiveTransform(self.corners, matrix)
```

Матрица переносит углы образца в текущий кадр. По четырём новым точкам можно нарисовать рамку с перспективой. В отличие от обычного прямоугольника по осям она повторяет наклон плоского объекта.

Далее проверяются конечность координат, выпуклость и площадь. Принимается площадь от 100 пикселей² до 1.5 площади кадра. Координаты не должны уходить дальше одной ширины или высоты за кадр, а видимая часть рамки должна занимать хотя бы 50 пикселей².

Проверки отбрасывают вырожденные, огромные и самопересекающиеся рамки. Они допускают частичный выход объекта за границы, но не доказывают, что рамка действительно точна. Порог площади задан для рабочих кадров проекта; для очень

маленьких объектов он может мешать обнаружению.

follow_points и резервный Lucas–Kanade

Метод используется, только если есть точки предыдущего успешного кадра. Изолированный кадр без состояния таким способом не обрабатывается.

```
current, forward, _ = cv2.calcOpticalFlowPyrLK(
    self.previous_gray, gray, self.previous_points, None, **params)
```

LK ищет новые положения старых точек. Окно 21 × 21 — локальная область поиска для этой операции. Это не размер объекта и не размер всего кадра. Пирамида с `maxLevel=3` помогает учитывать движение на нескольких масштабах. Критерий остановки ограничивает число итераций и требуемую точность.

Затем выполняется обратный перенос из текущего кадра в предыдущий. Если точка прошла путь туда и обратно надёжно, её обратная координата должна быть близка к исходной.

```
error = np.linalg.norm(backward - self.previous_points, axis=2).ravel()
good = (forward.ravel() == 1) & (reverse.ravel() == 1) & (error < 1.5)
```

Используются только точки с успешными статусами обоих переносов и обратной ошибкой меньше 1.5 пикселя. Нужны минимум 8 таких точек.

По парам `original_points → current` снова вычисляется гомография с RANSAC, теперь с допуском 3 пикселя. Требуются минимум 8 inliers и доля не меньше 0.6. Координаты первого набора находятся в системе постоянного образца, поэтому найденная матрица сразу переносит его исходные углы на текущий кадр.

locate и выбор способа сопровождения

1. Пробуется независимый SIFT-поиск образца.

2. Проверяется полученная рамка.

3. Если SIFT-оценка не принята, пробуется перенос LK и та же проверка геометрии.

4. Старые точки сбрасываются. Если есть принятая рамка, готовятся новые точки для следующего кадра.

5. Возвращаются рамка или `None`, а также статистика метода.

SIFT всегда пробуется первым. Поэтому он способен заново найти объект после потери и исправлять накопленный дрейф LK. Если оба метода не дали допустимую рамку, старая рамка не переносится автоматически.

Внутри найденного четырёхугольника создаётся маска, а `goodFeaturesToTrack` выбирает до 300 углов Shi–Tomasi с минимальным расстоянием 5 пикселей. Это точки для следующего переноса LK. Они не обязаны совпадать с точками SIFT.

```
inverse = np.linalg.inv(matrix)
original = cv2.perspectiveTransform(points, inverse)
```

Поскольку H переводит образец в кадр, обратная матрица переводит новые точки кадра обратно в систему образца. Сохраняются обе версии: текущая для LK и соответствующая исходная для следующего расчёта H. Это позволяет обновлять точки, не меняя систему координат рамки. Если матрица необратима или координаты некорректны, состояние LK не сохраняется.

При длительном сопровождении без успешного SIFT ошибка всё равно может накапливаться. Обновление точек не является полной защитой от дрейфа.

draw_result и изображение результата

Создаётся копия кадра, чтобы не рисовать поверх входных данных алгоритма. `polylines` соединяет четыре угла и замыкает рамку. Координаты округляются и преобразуются в `int32`, как требуется для рисования.

Подпись рисуется с тёмной обводкой, чтобы она читалась на разном фоне. В верхней полосе показываются статус, метод, число соответствий и inliers. При `corners=None` объектная рамка и подпись не рисуются. Подписи лучше задавать латиницей из-за ограничений стандартных шрифтов OpenCV.

run_video и полный цикл

`VideoCapture` открывает видео, затем первый кадр читается и при необходимости уменьшается. Образец берётся из выбранного ROI или из всего кадра. ROI проверяется: он должен помещаться внутри кадра и быть не меньше 16 × 16.

`VideoWriter` создаёт MP4 с кодеком `mp4v`, исходным FPS и рабочим размером кадра. Проверяется успешное открытие. Видео сохраняется без исходной звуковой дорожки: программа работает только с изображениями кадров.

Цикл обрабатывает и первый кадр, и все следующие. В нём вызывается `locate`, рисуется результат, записывается кадр и добавляется строка метрик. При `--show` кадр также выводится в окно. `Q` или `Esc` прерывает цикл и сохраняет только обработанную часть.

Время `processing_ms` измеряет только вызов `locate`. Отрисовка, чтение и запись видео, показ окна и подготовка образца находятся вне этого замера. `processing_fps = 1000 / mean_ms` — вычислительная скорость этого участка, а не FPS всего приложения. Сохранённый ролик воспроизводится с исходным FPS независимо от того, как долго шёл расчёт.

`metrics.csv` содержит строки для каждого обработанного кадра. `summary.json` хранит параметры, число кадров с рамкой, распределение методов и время. `time_s = frame / fps` — расчётное время по номеру кадра и FPS; для видео с переменным FPS оно может отличаться от точных исходных временных меток.

Ресурсы закрываются в блоке `finally`: файл видео и writer освобождаются даже при ошибке. Иначе выходной MP4 мог бы не завершиться корректно.

main и аргументы командной строки

`main()` создаёт парсер аргументов и вызывает `run_video`. Группа аргументов запрещает одновременное использование `--select` и `--roi`, поскольку это два способа задать одну область. Ошибки файла, параметров и OpenCV выводятся в терминал с ненулевым кодом завершения.

```
python main.py videos/my_video.mp4 --select --show --label My_Object --output output_my_video
```

Условие `if __name__ == "__main__"` запускает парсер только при прямом запуске файла. Когда `experiment.py` импортирует `run_video`, командная строка из `main.py` повторно не обрабатывается.

4. Зачем нужен experiment.py

`main.py` обрабатывает один выбранный ролик. `experiment.py` автоматизирует отчёт: последовательно запускает те же функции на трёх видео, собирает общую таблицу и строит изображения. Отдельной копии алгоритма трекинга в нём нет.

Подготовка

`ROOT = Path(__file__).resolve().parent` задаёт папку проекта независимо от текущей папки терминала. В `VIDEOS` перечислены три тестовых имени. `matplotlib.use("Agg")` позволяет сохранять графики без открытия окон.

По умолчанию вызывается `run_video` для каждого ролика. С `--reuse` читаются уже существующие `summary.json`, `metrics.csv` и выходные видео. Это перестроение материалов, а не новые измерения. Проверяется соответствие количества строк CSV числу обработанных кадров JSON.

make_examples

Открывается готовое `tracked.mp4`, из него берутся кадры 0, 100, 200, 280, 420 и 550. Последний индекс ограничивается концом ролика. Получается сетка 2 × 3 со статусом и рамкой, сохранённая в `assets`. Именно эти изображения вставлены в README.

Кадры выбраны заранее по номерам, а не только по удачным обнаружениям. Они показывают несколько масштабов и могут включать потери. Полное поведение нужно смотреть в выходном видео и покадровой таблице.

Общая таблица

Для каждого ролика берутся число кадров, число принятых оценок, процент, количество SIFT- и LK-кадров и время. Процент равен `100 * found_frames / frames`.

```
with (ROOT / "results.csv").open("w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(f, fieldnames=fields, extrasaction="ignore")
    writer.writeheader()
    writer.writerows(summaries)
```

Режим `w` создаёт или перезаписывает файл. `writeheader` записывает названия столбцов, `writerows` — три строки результатов. `extrasaction="ignore"` позволяет выбрать только нужные поля из более полной JSON-сводки.

Отдельно сохраняется `environment.json` с версиями Python, OpenCV, NumPy и названием платформы. Это помогает объяснить различия времени на другом компьютере.

Графики

`summary.png` содержит два графика. Слева доля кадров с рамкой разделена на SIFT и LK; справа показано среднее время `locate` в миллисекундах. `timeline.png` показывает выбранный метод на временной шкале: потеря, LK или SIFT.

Число кадров с принятой оценкой не является измеренной точностью границ. Без покадровой разметки нельзя достоверно посчитать IoU или процент правильных рамок. Доля inliers тоже характеризует согласованность точек, а не является вероятностью правильного обнаружения.

5. Тестирование и ограничения

`test_tracker.py` запускается стандартным `unittest`. Проверяются совпадение образца с самим собой, известная перспектива с ошибкой углов меньше 4 пикселей, резервный LK при запрещённом SIFT, потеря на чёрном кадре и восстановление, пустой образец, изменение размера и отсутствующий файл.

В тесте LK число требуемых SIFT-соответствий специально увеличивается до недостижимого значения после инициализации. Это приём проверки резервной ветки, а не настройка рабочего запуска.

```
python -m unittest -v test_tracker
python experiment.py
```

При обычной работе основные ограничения — сильное размытие, малый размер, блики, закрытие объекта, резкие скачки камеры и неплоская поверхность. В сомнительных кадрах рамка может исчезнуть. При длительном LK может появиться дрейф даже при принятой геометрии.

Для своего видео на первом кадре покажите плоский объект целиком и выделите его границы. Команды для съёмки, демонстрации и устранения ошибок приведены в `RUNBOOK.md`, короткая речь и ответы преподавателю — в `DEFENSE.md`.

Короткий порядок демонстрации

Откройте терминал в папке lr2. Создайте окружение и установите requirements.txt по инструкции RUNBOOK.md.

Запустите тестовый ролик с показом окна:

```
python main.py videos/mona-lisa.avi --show --label Mona_Lisa
```

```
python -m unittest -v test_tracker
```

```
python experiment.py
```

После съёмки собственного ролика выделите объект на первом кадре:

```
python main.py videos/my_video.mp4 --select --show
    --label My_Object --output output_my_video
```

Последняя команда показана с переносом для чтения: в терминал введите её одной строкой. На предоставленных роликах можно использовать весь первый кадр. Для собственного видео выделите объект целиком и подтвердите Enter. На первом кадре лучше снять его без сильного наклона.

Что показывать преподавателю

1. README и образец template.png.

2. Видео с зелёной рамкой и статусами sift / flow.

3. Методы match_template, follow_points, locate в main.py.

4. Результат семи автоматических проверок.

5. Графики summary.png и timeline.png, таблицу results.csv.

6. Свой ролик и его результаты, когда он будет снят.

Собственное видео ещё не проверено. Доля кадров с рамкой не равна точности границ. При повторных замерах время изменится; сохраняйте согласованную версию таблицы, графиков и текста отчёта.